High-Level Structure of Dependency Networks

Alexei Vazquez
Nodes & Links Ltd, Salisbury House, Station Road, Cambridge, CB1 2LA, UK
alexei@nodeslinks.com

Abstract

Dependency networks describe how activities, events or processes are linked by prerequisite relationships, but real networks are often too large to interpret directly. Here we ask how to summarise such a network at a higher level of organisation, using construction project schedules as a case study. We introduce two simple, deterministic ways of grouping activities: the Dependency Uprise Structure (DUS), which collects activities that share the same logical depth in the network, and the Dependency Predecessor Structure (DPS), which refines this grouping by also recording which earlier levels each activity depends on. Across 1198 construction project schedules from three companies, spanning 36 to 150,692 dependencies, the number of DUS levels grows only logarithmically with the number of dependencies, while the number of DPS classes grows as a sub-linear power law with exponent 0.69 (95% CI $[0.68, 0.71]$). Model selection favours these two functional forms over the alternatives we considered, and the logarithmic coefficient and the power-law exponent are statistically indistinguishable across the three companies. The DUS therefore compresses the dependency network exponentially and the DPS provides an intermediate, sub-linear summary. In both cases the mean distance from a DUS level to the end of the project is almost independent of project size, because dependencies that skip levels act as shortcuts. We replicate the qualitative results on the publicly available PSPLIB benchmark. Because dependency networks appear in fields as diverse as project management, causal inference, ancestry analysis and neural networks, the same approach should generalise beyond construction. A Python implementation is available at `https://github.com/av2atgh/dus`.

Introduction

Most large human endeavours are organised as projects [1, 2, 3]. A building is constructed, a piece of software is delivered, or a new drug is brought through clinical trials by coordinating many smaller activities under shared deadlines and budgets. Although the products are very different, the underlying logistics are remarkably similar, and the discipline of project management has emerged to plan and control them across sectors.

Coordinating a project of any size requires a high-level framework that connects day-to-day execution with strategic goals [4, 5]. Almost universally, this is achieved by breaking the total work into smaller, manageable units that can be assigned to teams, costed, scheduled and tracked. The way the work is broken down determines how progress is monitored and how risks are spotted.

The standard tool for this decomposition is the Work Breakdown Structure (WBS) [4]. The WBS is a tree of deliverables: the final product sits at the top and is recursively divided into ever smaller pieces of work, until the leaves are activities small enough to be scheduled and budgeted individually. Figure 1A shows a deliberately small example for a house: the project divides into substructure, superstructure and services, and these divide in turn into the individual activities that will appear on the schedule. Real construction WBSs follow exactly this pattern but reach tens of thousands of leaves. The resulting hierarchy is the backbone of project planning, organising responsibilities and providing a common map for everyone involved.

The WBS captures what needs to be done, but not the order in which it has to happen. To turn a list of activities into a schedule, the project manager must specify which activities are prerequisites for which others — the foundations of a building must be poured before the frame goes up, or a back-end service must be available before its front-end can be tested. The set of these prerequisite relationships forms a *dependency network* (Figure 1B). From this network one can compute the critical path, identify slack, and estimate when the project will end [6].

Formally, a dependency network is a directed graph [25] in which each node is an activity and each directed link encodes a prerequisite, with $A \to B$ meaning that activity A must finish before activity B can start. Such a network must be acyclic: a loop such as $A \to B \to C \to A$ would require A to be finished before it can begin. Dependency networks are therefore directed acyclic graphs (DAGs). Every DAG admits a *topological order*: an arrangement of its nodes in which all of an activity's predecessors come before it and all of its successors come after it. Some orderings are forced, as in the chain $A_1 \to A_2 \to \cdots \to A_n$, while others leave room for choice. If A_1 feeds both A_2 and A_2' and both must finish before A_3 starts, then A_2 and A_2' can be swapped without changing anything that matters for the schedule. Two nodes that can be swapped in this way are said to be *incomparable*: neither is an ancestor of the other. DAGs are the natural representation not only of schedules but also of causal models, ancestries and feedforward neural networks, which is why the constructions below are not specific to construction.

Project teams therefore look at their work through two different lenses: the deliverable-based hierarchy of the WBS and the prerequisite-based wiring of the dependency network. These two views rarely line up, because dependencies routinely cross WBS boundaries — in Figure 1 the utilities activity, which belongs to the services branch, depends on excavation, which belongs to the substructure branch. What is missing is a high-level summary of the dependency network itself — not a top-down decomposition imposed from above, but a structure that emerges from the dependencies. Recovering such a summary is an inverse problem: instead of starting with a goal and dividing it, we start with the fine-grained dependencies and try to expose their large-scale organisation.

Several literatures bear on this problem. In project management and engineering design, the Design Structure Matrix (DSM) represents dependencies as a square matrix and has a long tradition of reordering and clustering that matrix to reveal blocks of tightly coupled work and to sequence it [7, 8, 9]. DSM sequencing and the layering used here are close relatives: both seek an ordering compatible with the dependencies. They differ in aim. DSM methods are usually applied to problems that contain cycles, where the object is to find and break feedback loops, and clustering is typically the result of an optimisation with tunable objectives. The constructions below apply to acyclic networks, involve no optimisation and no free parameters, and are aimed at compression rather than resequencing. In computer science, condensing a directed graph by its strongly connected components [10] is the standard way of turning an arbitrary digraph into a DAG; it leaves an already-acyclic network untouched and so cannot compress a schedule. Transitive reduction [11] removes redundant edges but keeps every node. Layered graph drawing [26, 12] assigns nodes to layers by essentially the rule used below, but as a step towards a drawing rather than as an object of study. In network science, meso-scale structure is usually sought statistically, by inferring communities or blocks [20, 21, 22, 23, 24]; other approaches exploit exact symmetry [13, 14], graph fibrations [29, 30], latent hyperbolic geometry [15], or explicit measures of hierarchy [16, 17]. These methods have rarely been applied to dependency networks. As we show below, the dependency case is considerably simpler: a useful high-level structure can be extracted deterministically, without statistical inference and without free parameters, by exploiting the directional, acyclic nature of the network.

The rest of this paper defines the two groupings, illustrated in Figure 1, measures how they scale on 1198 real construction schedules, and asks how far apart the resulting coarse-grained networks are.

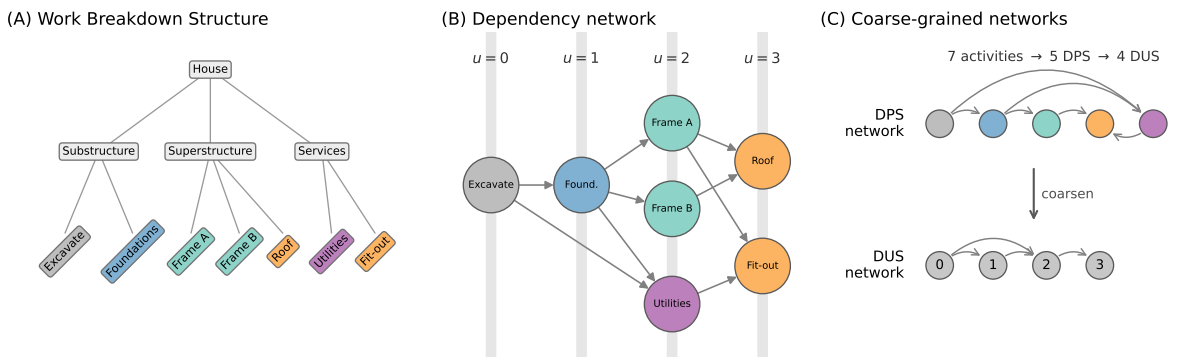

Figure 1: The two coarse-grainings, illustrated on a seven-activity project. (A) A Work Breakdown Structure: the project is recursively divided into deliverables until the leaves are schedulable activities. (B) The dependency network of those activities. The DUS index u of an activity is the length of the longest chain of dependencies leading to it, shown by the vertical bands. Note that the dependency from Excavate to Utilities crosses a WBS branch boundary. Colours indicate the DPS class, that is, the set of DUS levels the activity's predecessors occupy: Frame A and Frame B share a DUS level and a DPS class, whereas Utilities shares their DUS level but forms a separate DPS class because it also depends on level 0. (C) The two coarse-grained networks obtained by collapsing the dependency network onto DPS classes and onto DUS levels. Seven activities reduce to five DPS classes and four DUS levels.

Results

Dependency Uprise Structure

We assign every activity an integer that measures how many dependency steps separate it from a starting activity. We set the DUS index $u_i = 0$ for every activity with no predecessors, and otherwise

$$u_i = 1 + \max_{j \in \text{PredecessorsOf}(i)} u_j. \tag{1}$$

After processing all activities in a topological order, u_i equals the length of the longest chain of dependencies leading to activity i. We call it the *uprise* index because it counts how far the activity has risen above the start of the project, in dependency steps rather than in calendar time.

Three distinct objects follow from Eq. (1), and we keep them separate throughout. The *DUS index* u_i is an integer attached to each activity. A *DUS level* is the set of all activities sharing one value of u_i. The *Dependency Uprise Structure* (DUS) is the resulting partition of the activities into levels. The DUS network, defined in the next section, is a graph whose nodes are DUS levels.

By construction, two activities in the same DUS level are pairwise incomparable in the dependency DAG — neither is an ancestor of the other — so they can appear in either relative order within a valid topological sort. The converse does not hold: two activities at different depths can also be incomparable, so the DUS is one specific antichain decomposition of the DAG, not in general the coarsest one.

It is worth being explicit about what Eq. (1) is not. It is not a breadth-first search from the project start, which would assign each activity the length of the *shortest* path from a starting activity. The two differ whenever an activity has predecessors at different depths: in Figure 1B, a breadth-first search from Excavate would place Utilities at depth 1, because Excavate feeds it directly, whereas Eq. (1) places it at $u = 2$, because it must also wait for Foundations. Only the longest-path rule has the property that matters here, namely that all of an activity's predecessors lie strictly below it, so that the levels are ordered and every activity is preceded

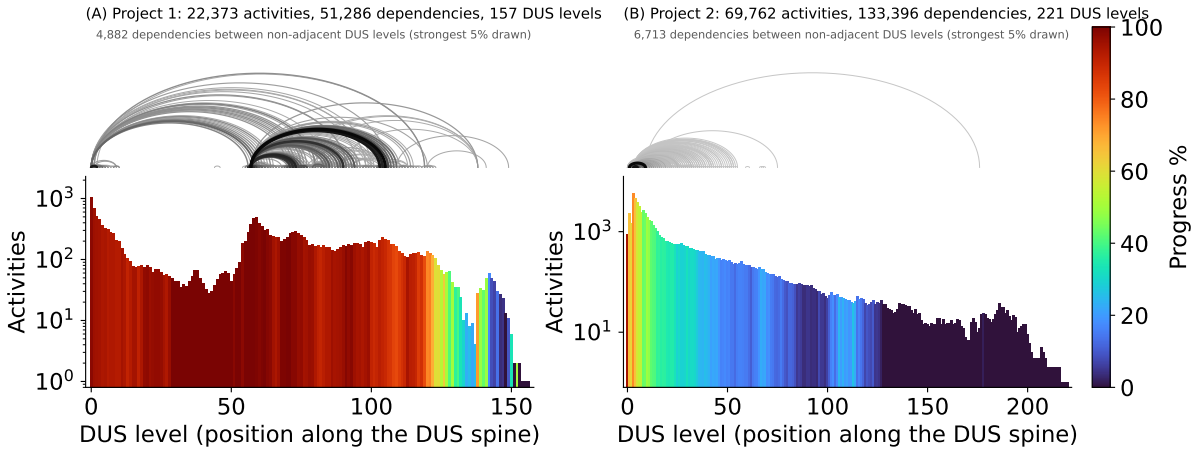

Figure 2: DUS profiles of two construction project schedules. The horizontal axis is position along the DUS spine. Bars give the number of activities in each DUS level and are coloured by the mean Physical Progress Percent of those activities, a standard progress metric in construction. Arcs above show dependencies between non-adjacent DUS levels, with darker and thicker arcs encoding more dependencies; only the strongest 5% are drawn, since drawing all of them obscures the structure. Both schedules are included in the public code repository.

by the entire history it depends on. It is also not a breadth-first search backwards from the project-completion node, which would group activities by their remaining depth to the end rather than by their accumulated depth from the start.

The construction in Eq. (1) is the layering used in the first phase of the Sugiyama framework for hierarchical graph drawing [26, 12] and is closely related to the Coffman–Graham scheduling algorithm [27]. In project management terms, the DUS index of the final activity is the number of activities on the longest dependency chain, which is the critical path when all activities have equal duration. The contribution of this paper is not the layering itself but its use as a coarse-graining of the dependency network, the empirical scaling relations established below, and the finer DPS construction introduced later.

The DUS network

Once activities have been grouped into DUS levels, we can collapse the original dependency network onto these groups. In the resulting DUS network each node is a DUS level, and a weighted edge $U_a \xrightarrow{w_{ab}} U_b$ records that w_{ab} activity-level dependencies run from activities in level a to activities in level b. The DUS network inherits the directed acyclic structure of the dependency network. What is new is the weight w_{ab}, which measures how strongly two DUS levels are coupled.

The simplest DUS network is a chain, for example $U_0 \xrightarrow{w_{01}} U_1 \xrightarrow{w_{12}} U_2$. In general, dependencies may also skip ahead, linking distant levels. A chain through every level is, however, always present: by Eq. (1), every activity has at least one predecessor in the previous DUS level, so $w_{i-1,i} \geq 1$. The chain $U_0 \to U_1 \to \cdots \to U_{m-1}$, where m is the number of DUS levels, is therefore the longest path in the DUS network. We call it the *DUS spine*, by analogy to the activity-level critical path while avoiding overlap with that term's specific duration-weighted meaning in project management.

Because the DUS network has at most a few hundred nodes even for the largest schedules, it is small enough to draw, which makes it a natural canvas on which to overlay other project information. Figure 2 shows what we call the *DUS profile* of two real construction schedules: the DUS spine runs along the horizontal axis, the height of each bar is the number of activities in that level, the colour encodes progress, and the arcs above record dependencies that skip levels.

Company	Projects	Activities	Dependencies	DUS levels	DPS classes
		median [minimum–maximum]			
Blue	82	43,167 [83–70,503]	73,649 [101–137,450]	170 [32–238]	3,745 [35–8,937]
Green	249	4,520 [37–104,981]	7,284 [36–150,692]	108 [17–400]	1,003 [21–8,177]
Orange	867	9,921 [37–64,706]	17,117 [55–129,067]	104 [14–571]	1,074 [19–8,365]
All	1198	7,942 [37–104,981]	13,713 [36–150,692]	108 [14–571]	1,063 [19–8,937]

Table 1: Descriptive statistics of the construction project schedules analysed here. The three companies are anonymised and identified by the colours used in Figures 3–5.

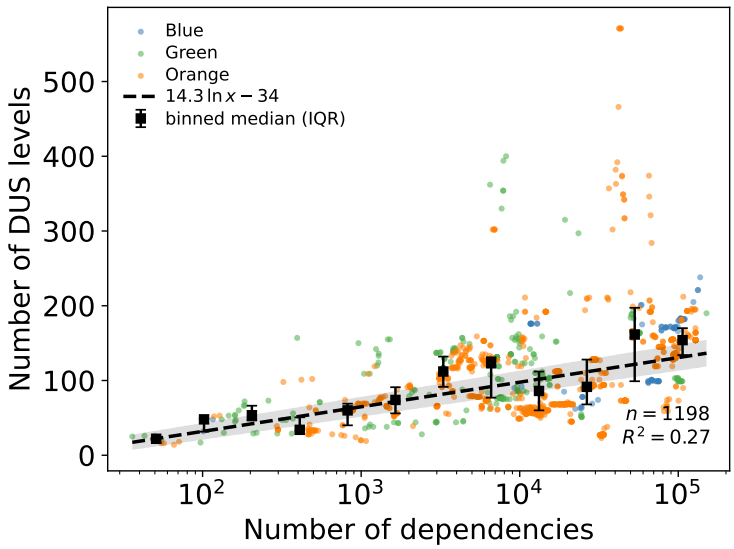

Figure 3: The number of DUS levels grows logarithmically with the number of dependencies. Each point is one of 1198 construction projects, coloured by company. The dashed line is the maximum-likelihood fit $m = p \ln x + c$ under multiplicative errors, with $p = 14.3$ and $c = -33.9$; the grey band is the 95% bootstrap confidence region. Black squares are binned medians with interquartile ranges.

Read this way, a 22,373-activity schedule becomes a single legible picture in which one can see where the work is concentrated, how far it has progressed, and where long-range coupling occurs.

Scaling properties

To probe how the DUS network scales with project size we collected 1198 schedules from three construction companies, identified as blue, green and orange (see Methods). Table 1 reports their descriptive statistics. The projects span 37 to 104,981 activities and 36 to 150,692 dependencies, close to four orders of magnitude, and the three companies overlap substantially in size.

The number of DUS levels grows very slowly with the number of dependencies (Figure 3). Because a logarithm and a power law with a small exponent are hard to tell apart over a finite range, we compared four functional forms — a one-parameter logarithm, a two-parameter logarithm, a power law and a linear relation — each fitted by maximum likelihood under both additive and multiplicative errors, and selected between them by AIC (Methods). The two-parameter logarithm with multiplicative errors is preferred, with $m = 14.3 \ln x - 33.9$ where x is the number of dependencies. It is favoured over the power law by $\Delta\mathrm{AIC} = 22$, over the one-parameter logarithm by $\Delta\mathrm{AIC} = 45$, and over the linear relation by $\Delta\mathrm{AIC} = 147$; its residuals show no detectable heteroscedasticity (Breusch–Pagan $p = 0.07$). The best-fitting power law has exponent 0.17, small enough that the practical distinction from a logarithm matters little

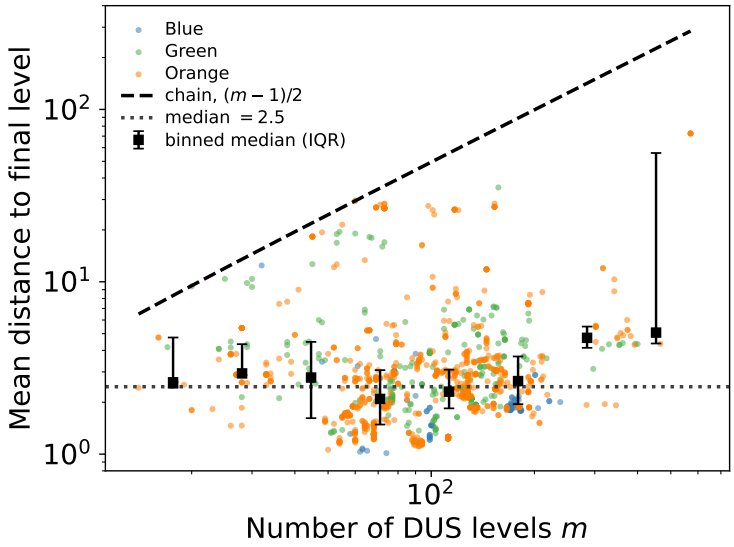

Figure 4: Dependencies that skip levels make the DUS network shallow. Mean shortest distance from a DUS level to the final DUS level, against the number of DUS levels m. The dashed line is the expectation $(m-1)/2$ for a pure chain. The distance is essentially independent of m, with median 2.5 (dotted line). Black squares are binned medians with interquartile ranges.

over the observed range. The logarithmic coefficient is 14.3 with a 95% bootstrap confidence interval of $[13.4, 15.2]$.

The relation is a statement about the central tendency, not about individual projects: it explains 27% of the variance in $\ln m$, and at a given number of dependencies the number of DUS levels varies by a factor of several. We therefore describe it as a slow, logarithmic growth of the central trend rather than as a scaling law in the strict sense. Even so, the compression it implies is substantial. Increasing the number of dependencies by a factor of ten adds about 33 DUS levels regardless of where one starts, so the DUS network is an exponential compression of the underlying dependency network. Concretely, the median project in Table 1 has 13,713 dependencies and 108 DUS levels.

Fitting each company separately gives logarithmic coefficients of 15.9, 16.5 and 14.6. A likelihood ratio test of a model with a shared coefficient and company-specific offsets against a model with fully company-specific parameters does not reject the shared coefficient (LR $=$ 2.4, df $= 2$, $p = 0.30$), and AIC prefers the shared-coefficient model. The offsets do differ significantly (LR $= 38.4$, df $= 2$, $p = 5 \times 10^{-9}$). So the rate at which levels accumulate with dependencies is common to the three companies at 15.1 per natural logarithm, while the absolute number of levels is shifted by company-specific scheduling practice.

A second question is how far apart, on average, two DUS levels are in the network. If a project had no dependencies outside the DUS spine, the mean shortest distance from a level to the final one would be exactly $(m-1)/2$. Real projects fall far below this expectation (Figure 4): the median distance is 2.5 levels, with an interquartile range of $[1.9, 3.6]$, and 99.8% of projects lie below the chain expectation, at a median of 4.6% of it. More striking than the gap is the near-absence of growth. Binned medians are 3.4 for projects with fewer than 50 levels, 2.3 for 50 to 150 levels, and 2.7 above 150 levels, even though m ranges from 14 to 571. The rank correlation between the two is small though nominally significant ($\rho = 0.10$, $p = 8 \times 10^{-4}$, $n = 1198$). Applying the model selection procedure used above, a weakly increasing linear relation is in fact preferred over a strictly constant one (ΔAIC $= 20$), with slope 0.0041 per level (95% CI $[0.0019, 0.0062]$). The growth this implies is negligible in practice: across the entire observed range the fitted mean distance rises only from 2.5 to 4.7, a factor of 1.9, while

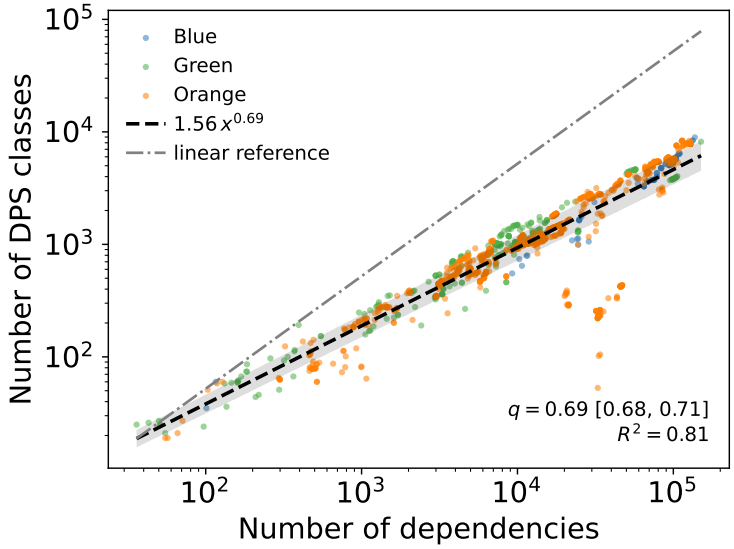

Figure 5: The number of DPS classes grows as a sub-linear power law with the number of dependencies. Each point is one of 1198 construction projects, coloured by company. The dashed line is the maximum-likelihood fit $1.56\,x^{0.69}$ under multiplicative errors; the grey band is the 95% bootstrap confidence region. The dash-dotted line is a linear reference of slope 1.

the number of levels grows by a factor of 41.

The reason is that many dependencies act as shortcuts, linking levels that are far apart along the spine (see the arcs in Figure 2). This is the mechanism that produces short paths in small-world networks [28], and it answers a question a project manager might ask directly: a schedule with 500 DUS levels is not 500 steps deep in any practical sense, because a handful of long-range dependencies carry information from almost anywhere to the end in two or three steps. We stop short of calling these networks small-world in the Watts–Strogatz sense. That definition requires both short path lengths and clustering substantially above a degree-preserving random null, and the standard clustering coefficient is not well defined for a DAG, in which triangles cannot close. Establishing a clustering criterion appropriate to DAGs, and the right null model to compare it against, is beyond the scope of this paper. What the data support is the shorter statement: shortcuts make the DUS network shallow, and its depth does not grow appreciably with project size.

DUS fine structure

Activities sharing the same DUS index can be split further according to which DUS levels their predecessors come from (Figure 1B). An activity at $u = i$ must have at least one predecessor at $u = i - 1$; predecessors at $u = 0, \ldots, i - 2$ are optional. Each of the $i - 1$ optional levels can be either present or absent, giving 2^{i-1} distinguishable predecessor patterns at the i-th level. Activities at $u = 0$ have no predecessors, and those at $u = 1$ draw exclusively from $u = 0$, so neither can be split further.

We call the resulting partition the *Dependency Predecessor Structure* (DPS) of the network, and each of its parts a DPS class. As with the DUS, we distinguish the DPS index of an activity, the DPS class as a set of activities, the DPS as the partition, and the DPS network defined below. These groupings are related to what mathematicians call *graph fibrations* [29, 30]: two activities in the same DPS class have predecessor sets that occupy the same levels, a local symmetry of the kind that fibrations formalise and that has been used to find building blocks in biological networks. Restricting to DAGs simplifies matters, because each class can be labelled

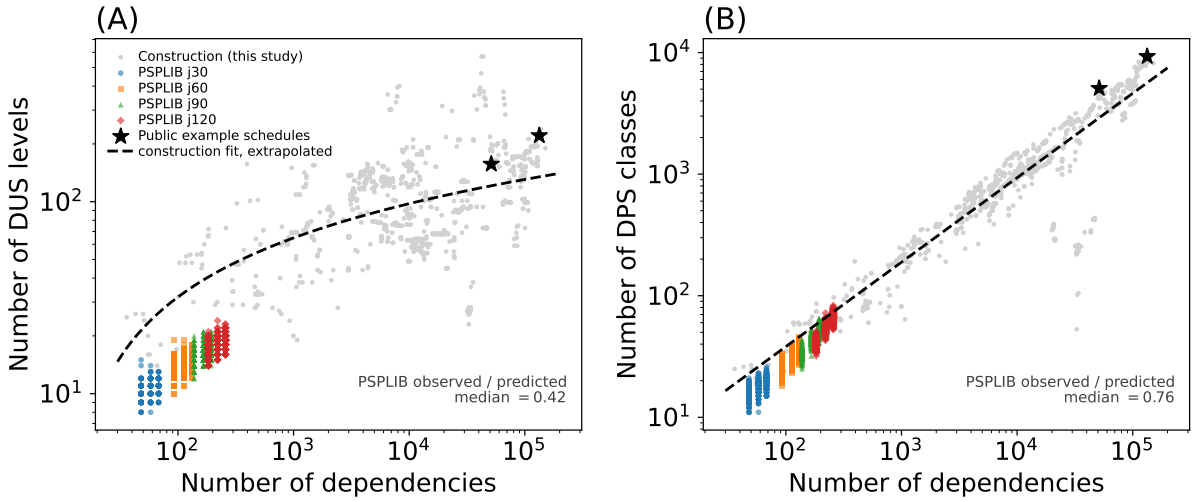

Figure 6: Replication on public benchmark instances. (A) Number of DUS levels and (B) number of DPS classes against the number of dependencies, for 2040 single-mode PSPLIB instances (coloured), the 1198 construction schedules of this study (grey), and the two anonymised real schedules distributed with the code repository (stars). Dashed lines are the fits to the construction data, evaluated two orders of magnitude below the median project size on which they were estimated. Only 2.4% of the construction projects are as small as the largest PSPLIB instance.

by a single integer. We set $f_i = 0$ when $u_i = 0$ and otherwise

$$f_i = \sum_{k=0}^{u_i-2} b_{ik}\, 2^k + 2^{u_i-1} \tag{2}$$

where $b_{ik} = 1$ if activity i has at least one predecessor in DUS level k and $b_{ik} = 0$ otherwise. The integer f_i should be read as a categorical label, not a metric: numerical proximity of two f_i values does not imply similarity of their predecessor patterns.

As before, we collapse the dependency network onto its DPS classes to obtain the DPS network: each node is a DPS class, and a weighted edge $F_a \xrightarrow{w_{ab}} F_b$ records the w_{ab} activity-level dependencies between them. Figure 5 plots the number of DPS classes in each project against the number of dependencies. Applying the same model selection procedure as above, a power law under multiplicative errors is preferred, with $1.56\,x^{0.69}$, ahead of the linear relation by $\Delta\mathrm{AIC} = 314$ and the two-parameter logarithm by $\Delta\mathrm{AIC} = 1328$. The exponent is 0.694 with a 95% bootstrap confidence interval of $[0.679, 0.709]$, comfortably below 1, and the fit explains 81% of the variance in the logarithm. The residuals of this fit are heteroscedastic, with more scatter at large sizes, so the confidence interval quoted here is a case-resampling bootstrap rather than a nominal standard error. As for the DUS, a shared exponent across the three companies is not rejected ($\mathrm{LR} = 1.4$, $\mathrm{df} = 2$, $p = 0.50$; shared exponent 0.71), while the prefactors differ ($p = 4 \times 10^{-7}$).

The growth is therefore sub-linear but much faster than logarithmic, placing the DPS network between the original dependency network and the much smaller DUS network. The theoretical bound 2^{i-1} on the number of patterns at level i is loose in practice by an enormous margin: a project with 100 DUS levels could in principle realise more than 10^{29} classes and instead realises of order 10^3. Most predecessor patterns that are combinatorially possible simply do not occur in real schedules.

Replication on public data

The construction schedules analysed here are proprietary. To make the central results reproducible from public data, we repeated the analysis on the 2040 single-mode instances of PSPLIB [19], the standard benchmark library for resource-constrained project scheduling, and on the two anonymised real schedules distributed with our code (Figure 6).

PSPLIB instances are much smaller than real construction schedules. They contain 32 to 122 activities and 48 to 257 dependencies, come in only four activity counts, and span barely half a decade in size; their dependency range overlaps only the extreme lower tail of ours, since just 2.4% of our construction projects (29 of 1198) are that small. Model selection within PSPLIB alone favours a power law for the number of DUS levels and a linear relation for the number of DPS classes, which are not the forms selected on the construction data. Over so narrow a range, and with only four distinct sizes, we do not read this as evidence about functional form. We use PSPLIB instead for the question it can answer, which is whether the construction relations transfer to independent data at a size where they were barely constrained.

For the DPS the answer is yes. The construction power law, evaluated two orders of magnitude below the median project on which it was estimated, predicts the PSPLIB values to a median observed/predicted ratio of 0.76 (Figure 6B), and the power law fitted within PSPLIB alone, though not the form AIC prefers there, has exponent 0.86, also sub-linear. For the DUS the answer is no: the construction logarithm over-predicts the PSPLIB values by more than a factor of two (median observed/predicted ratio 0.42, Figure 6A). The logarithmic relation should therefore be read as a description of large construction schedules rather than as a universal law, which is a further reason to avoid the term scaling law for it. The qualitative claims do transfer: PSPLIB DUS networks are also shallow, with a median distance to the final level of 2.1, below the chain expectation in 100% of instances.

Discussion

Dependency networks admit two natural higher-level representations forming a chain of progressively coarser views, $\mathit{Activity} \to \mathit{DPS} \to \mathit{DUS}$. For construction project schedules, the DPS provides a sub-linear coarse-graining and the DUS an exponential one.

The practical value of these constructions to project management is that they give a view of the schedule that the WBS cannot. A WBS reports progress by deliverable; the DUS reports it by logical depth, which is what determines what can start next. Three uses follow directly. First, reporting and tracking: the DUS profile of Figure 2 compresses a schedule of tens of thousands of activities into a picture with of order a hundred nodes, on which progress, cost or risk can be overlaid, and in which a stalled level or an unusually large concentration of work is immediately visible. Second, diagnosis: the arcs in Figure 2 are dependencies that jump many levels, and these are exactly the links whose delay propagates furthest; they are candidates for scrutiny because a long-range dependency is often either a genuine structural constraint or an artefact of how the schedule was built. Third, as an input to risk models, since a coarse-grained network with a hundred nodes is tractable for simulation where one with a hundred thousand is not. That the compression is deterministic and parameter-free matters here: two planners applying it to the same schedule obtain the same answer, which is not true of clustering methods that depend on an objective function or a random seed.

Three limitations should be kept in view. The relations reported here are established on schedules from three companies in one sector, and the failure of the logarithmic relation to extrapolate to PSPLIB shows that the coefficients are not universal. The DUS is one antichain decomposition among many, and we have not shown it is optimal by any criterion; it is justified by being canonical, cheap and interpretable rather than by an optimality argument. Finally, the shortcut result is a statement about distances only, and we have deliberately not claimed

the full small-world property, which would require a notion of clustering appropriate to DAGs.

Because the construction is purely topological, the same machinery applies to any system representable as a directed acyclic graph, including causal networks, ancestries and feedforward neural networks. Three questions follow. Do the size relations observed for construction schedules carry over to those settings, and if the coefficients differ, what do the differences measure? What features of the underlying dependency network determine the structure of its DPS and DUS networks? And what is the right definition of clustering in a DAG, which would allow the shortcut result reported here to be sharpened into a proper small-world claim?

Methods

Dataset

The construction project schedules analysed here were drawn from the Nodes & Links database. Three construction companies are represented, identified throughout as blue, green and orange, contributing 82, 249 and 867 projects respectively, for a total of 1198. Table 1 reports the number of activities, dependencies, DUS levels and DPS classes for each company. One further schedule was excluded because its first DUS level connects directly to every other level, so that its DUS network is a star rather than a network with a spine. Activity-level dependencies and Physical Progress Percent values were taken directly from the schedules as recorded in the database.

The public replication uses the four single-mode PSPLIB instance sets j30, j60, j90 and j120 [19], comprising 2040 instances of 32, 62, 92 and 122 activities, downloaded from the PSPLIB distribution site. The precedence relations of each instance were extracted and treated exactly as the dependencies of a construction schedule.

Computation of the DUS and DPS

The DUS index u_i defined in Eq. (1) is computed by first sorting the activities into a topological order using Kahn's algorithm, then sweeping through that order and applying Eq. (1) to each activity in turn. The cost is linear in the number of activities plus the number of dependencies. The DPS index f_i defined in Eq. (2) requires a single additional pass over the dependencies once u_i is known: for each activity i we record which lower DUS levels its predecessors occupy, and combine these bits into the integer label given by Eq. (2). The DUS and DPS networks are then obtained by collapsing the dependency network onto these labels and accumulating edge weights. Distances within the DUS network are shortest-path distances in the unweighted directed network, computed by a single reverse sweep over the levels.

Statistical methods

For each relation we fitted four candidate functional forms: a one-parameter logarithm $y = p \ln x$, a two-parameter logarithm $y = p \ln x + c$, a power law $y = ax^b$, and a linear relation $y = ax + b$. Each was fitted by maximum likelihood under two error structures: additive, $y = f(x) + \varepsilon$ with $\varepsilon \sim N(0, \sigma^2)$, and multiplicative, $y = f(x)\, e^{\varepsilon}$ with $\varepsilon \sim N(0, \sigma^2)$. The multiplicative log-likelihoods include the Jacobian term $-\sum_i \ln y_i$, so that all eight fits are likelihoods of the same observed quantity and their AIC and BIC values are directly comparable across both functional form and error structure. This addresses the fact that a naive ordinary least squares fit on log-log axes is not comparable, on its own, to a fit on the natural scale.

Confidence intervals are nonparametric case-resampling bootstraps with 2000 replicates, which do not assume homoscedastic residuals; this matters for the DPS relation, whose residuals are heteroscedastic. Heteroscedasticity was assessed with a Breusch–Pagan test of the squared residuals against $\ln x$.

Commonality of parameters across companies was assessed with nested likelihood ratio tests under the selected error structure, comparing three models: a pooled model with one shared parameter pair; a model with a shared logarithmic coefficient, or power-law exponent, and company-specific offsets or prefactors; and a model with fully company-specific parameters. This separates the question of whether the companies share the form of the relation from the question of whether they share its nuisance parameters. We report the likelihood ratio statistic, its degrees of freedom and its p value, together with the AIC of each model.

We did not apply the Clauset–Shalizi–Newman procedure [18], which is designed to fit and test heavy-tailed probability distributions of a single variable. The relations studied here are regressions of one measured quantity on another across projects, not distributions, so that procedure does not apply; the analogous concerns it raises about naive log-log fitting are addressed by the explicit error-model comparison and bootstrap intervals described above.

Code and data availability

A Python implementation of the DUS and DPS algorithms, together with the two anonymised construction schedules used in Figure 2 and helper code for the DUS profile plots, is available at `https://github.com/av2atgh/dus`. The scripts that perform the statistical analysis, download and process the PSPLIB benchmark, and generate every figure in this paper are included in the same repository. The PSPLIB instances are publicly available from `https://www.om-db.wi.tum.de/psplib/`. The construction project schedules are proprietary to Nodes & Links Ltd and its clients and cannot be released; Table 1 reports their summary statistics, and Figure 6 shows that the main results are reproducible on public data.

Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. Nodes & Links Ltd provided support in the form of salary for Alexei Vazquez but did not have any additional role in the conceptualisation of the study, the analysis, the decision to publish or the preparation of the manuscript.

Author contributions

A.V. conceived the study, designed and implemented the algorithms, performed the analysis, prepared the figures and wrote the manuscript.

Use of generative AI

The author used Claude (Anthropic) for language editing of the manuscript text and for assistance with the statistical analysis code and figure generation scripts. The tool was not used for the conception of the study or the design of the algorithms. All analyses were verified by the author, and all scientific content, claims and conclusions are the responsibility of the author.

References

[1] Anders Jensen, Christian Thuesen, and Joana Geraldi. The Projectification of Everything: Projects as a Human Condition. *Proj. Manag. J.*, 47(3):21–34, 2016.

[2] Yvonne-Gabriele Schoper, Andreas Wald, Helgi Thor Ingason, and Thordur Vikingur Fridgeirsson. Projectification in western economies: A comparative study of germany,

norway and iceland. *International Journal of Project Management*, 36(1):71–82, 2018. Festschrift for Professor J. Rodney Turner.

[3] Reinhard Wagner, Martina Huemann, and Mladen Radujkovic. The influence of project management associations on projectification of society – an institutional perspective. *Project Leadership and Society*, 2:100021, 2021.

[4] Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*. Project Management Institute, Newtown Square, PA, seventh edition, 2021.

[5] Lars Taxén. *The System Anatomy – Enabling Agile Project Management*. Studentlitteratur, Lund, 2011.

[6] James E. Kelley Jr. and Morgan R. Walker. Critical-path planning and scheduling. In *Papers Presented at the December 1–3, 1959, Eastern Joint IRE-AIEE-ACM Computer Conference*, pages 160–173, New York, 1959. ACM Press.

[7] Donald V. Steward. The design structure system: A method for managing the design of complex systems. *IEEE Transactions on Engineering Management*, EM-28(3):71–74, 1981.

[8] Tyson R. Browning. Applying the design structure matrix to system decomposition and integration problems: A review and new directions. *IEEE Transactions on Engineering Management*, 48(3):292–306, 2001.

[9] Steven D. Eppinger and Tyson R. Browning. *Design Structure Matrix Methods and Applications*. MIT Press, Cambridge, MA, 2012.

[10] Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.

[11] A. V. Aho, M. R. Garey, and J. D. Ullman. The transitive reduction of a directed graph. *SIAM Journal on Computing*, 1(2):131–137, 1972.

[12] Patrick Healy and Nikola S. Nikolov. Hierarchical drawing algorithms. In Roberto Tamassia, editor, *Handbook of Graph Drawing and Visualization*, pages 409–453. Chapman and Hall/CRC, Boca Raton, FL, 2013.

[13] Ben D. MacArthur, Rubén J. Sánchez-García, and James W. Anderson. Symmetry in complex networks. *Discrete Applied Mathematics*, 156(18):3525–3531, 2008.

[14] Rubén J. Sánchez-García. Exploiting symmetry in network analysis. *Communications Physics*, 3:87, 2020.

[15] Dmitri Krioukov, Fragkiskos Papadopoulos, Maksim Kitsak, Amin Vahdat, and Marián Boguñá. Hyperbolic geometry of complex networks. *Phys. Rev. E*, 82:036106, Sep 2010.

[16] Enys Mones, Lilla Vicsek, and Tamás Vicsek. Hierarchy measure for complex networks. *PLoS ONE*, 7(3):e33799, 2012.

[17] Bernat Corominas-Murtra, Joaquín Goñi, Ricard V. Solé, and Carlos Rodríguez-Caso. On the origins of hierarchy in complex networks. *Proceedings of the National Academy of Sciences*, 110(33):13316–13321, 2013.

[18] Aaron Clauset, Cosma Rohilla Shalizi, and M. E. J. Newman. Power-law distributions in empirical data. *SIAM Review*, 51(4):661–703, 2009.

[19] Rainer Kolisch and Arno Sprecher. PSPLIB — a project scheduling problem library. *European Journal of Operational Research*, 96(1):205–216, 1997.

[20] Filippo Radicchi, Claudio Castellano, Federico Cecconi, Vittorio Loreto, and Domenico Parisi. Defining and identifying communities in networks. *Proceedings of the National Academy of Sciences*, 101(9):2658–2663, 2004.

[21] M. E. J. Newman. Modularity and community structure in networks. *Proceedings of the National Academy of Sciences*, 103(23):8577–8582, 2006.

[22] Jake M. Hofman and Chris H. Wiggins. Bayesian approach to network modularity. *Phys. Rev. Lett.*, 100:258701, Jun 2008.

[23] Brian Karrer and M. E. J. Newman. Stochastic blockmodels and community structure in networks. *Phys. Rev. E*, 83:016107, Jan 2011.

[24] Tiago P. Peixoto. Network reconstruction via the minimum description length principle. *Phys. Rev. X*, 15:011065, Mar 2025.

[25] Jonathan L Gross and Jay Yellen. *Graph Theory and Its Applications*. Chapman and Hall/CRC, Boca Raton, FL, 2nd edition, 2005.

[26] Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiko Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125, 1981.

[27] E. G. Coffman and R. L. Graham. Optimal scheduling for two-processor systems. *Acta Informatica*, 1(3):200–213, 1972.

[28] Duncan J. Watts and Steven H. Strogatz. Collective dynamics of 'small-world' networks. *Nature*, 393(6684):440–442, June 1998.

[29] Paolo Boldi and Sebastiano Vigna. Fibrations of graphs. *Discrete Mathematics*, 243(1):21–66, 2002.

[30] Flaviano Morone, Ian Leifer, and Hernán A. Makse. Fibration symmetries uncover the building blocks of biological networks. *Proceedings of the National Academy of Sciences*, 117(15):8306–8314, 2020.